Informe Proyecto 3: Del Pochoco SoC a un macro ASIC

Arquitectura de Computadores · Semestre 2026-2 · Universidad de los Andes · Facultad de Ingeniería y Ciencias Aplicadas

Profesor: Jorge Gómez Mir

Integrantes: Lucas Fernández · Maximiliano Ferrando · Renato Rivadeneira · Matías Ureta

Repositorio de trabajo: Proyecto3_ASIC_Grupo7

Documento preparado siguiendo el formato del informe del Proyecto 2. Versión de trabajo: resultados RTL verificables y tablas para completar con LibreLane.

1. OBJETIVO Y ALCANCE

El Proyecto 3 consiste en adaptar el Pochoco SoC, que en el Proyecto 2 ejecutaba un juego de reflejos en la Go Board, para obtener un macro digital en tecnología ASIC. Usamos LibreLane y el PDK GF180MCU. El objetivo no es programar una FPGA, sino llegar desde RTL hasta un diseño físico GDSII, comparar dos opciones de memoria y justificar las decisiones con reportes medidos.

Se conserva el Espino Core RV32E y el programa del juego en Assembly. Lo que cambia principalmente es la forma de implementar la memoria, el reset y la interfaz de entrada/salida. Los pines de asic_top son terminales lógicos del macro; no representan pads con protección ESD ni drivers de potencia.

1.1. Cumplimiento de los requisitos del enunciado

Requisito	Implementación y estado	Sección
Adaptar RTL FPGA a ASIC	asic_top y módulos ASIC implementados; faltan logs citados	2
Comparar dos memorias	ROM + RAM FF y memoria unificada FF implementadas; faltan métricas	3
Síntesis y módulos	Scripts y preservación de jerarquía listos; faltan reportes finales	4
Timing: barrido, ruta y mejora	Barrido/optimización preparados; no hay resultados físicos adjuntos	5
Floorplan y costo	Pin ordering y scripts listos; faltan 3+ corridas	6
Signoff y GDSII	Requiere reportes DRC, LVS, antena y capturas	7
RTL vs post-síntesis	Testbench y estímulos preparados; faltan logs reproducibles	8

PENDIENTE DE EVIDENCIA REAL: El informe todavía es borrador. No hay evidencia de cierre físico en el repositorio compartido, por lo que no se afirma que DRC/LVS/timing hayan aprobado.

2. DISEÑO GENERAL Y ADAPTACIÓN FPGA → ASIC

2.1. Arquitectura del proyecto

Separamos el diseño en tres carpetas. rtl/common reúne los módulos compartidos (core, periféricos, SPI y contador). rtl/fpga mantiene las partes específicas de la Go Board. rtl/asic contiene el top-level y las alternativas de memoria y register file adecuadas al flujo ASIC. Así evitamos modificar el core por decisiones que solo pertenecen a una tecnología.

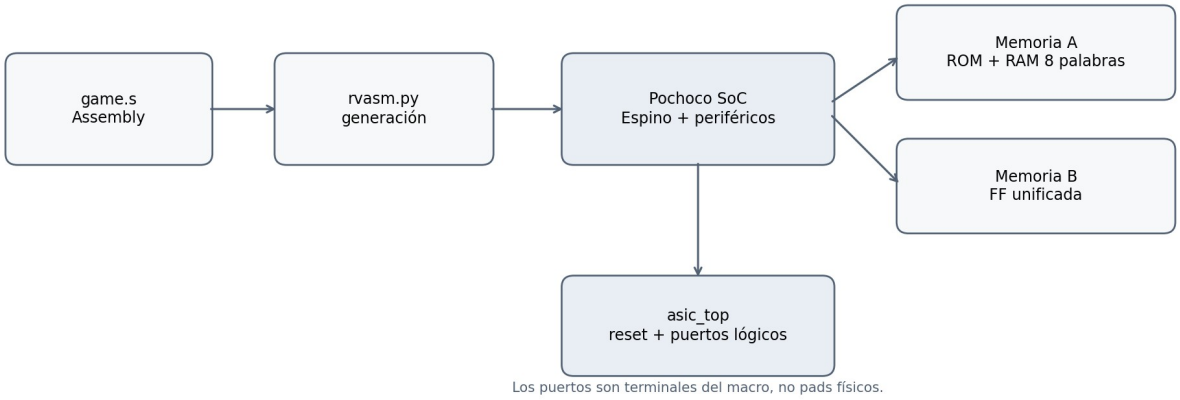


Figura 1. Flujo lógico del proyecto: el assembler entrega el programa y se elige una de las dos memorias ASIC.

2.2. Jerarquía y módulos principales

Módulo o archivo	Función
asic_top.v	Top-level ASIC; puertos lógicos de reloj, reset, LEDs, botones, displays y SPI
pochoco_reset_sync.v	Reset activo en bajo, con liberación sincronizada
pochoco_soc.v	Interconecta el core, memoria y periféricos
espino_register_file_ff.v	Banco RV32E implementado con FF y lectura registrada
pochoco_cycle_counter.v	Contador de 32 bits accesible por software
pochoco_mem_rom.v / pochoco_mem_ff.v	Estrategias A y B de almacenamiento
game_rom.v y game_mem.vh	Información generada automáticamente desde game.s

2.3. Cambios de FPGA a ASIC

Construcción en Proyecto 2	Por qué no corresponde al macro	Cambio adoptado
SB_RAM40_4K del iCE40	Primitiva específica de FPGA	FF y multiplexores en el register file ASIC
$readmemh / BRAM	No existe archivo game.hex al encender un chip	ROM combinacional o FF inicializados por reset
Valor inicial al encender	No garantiza estado físico de FF ASIC	Reset externo más sincronizador
Reloj fijo 25 MHz	La entrega ASIC fija CLOCK_PERIOD = 100 ns	Constante CLK_HZ de 10 MHz al ensamblar
Polaridad display Go Board	Depende del hardware de la placa	Señales lógicas positivas en el macro

El código identifica estas diferencias, pero el enunciado también pide asociar cada construcción al warning/error de LibreLane, indicando archivo y línea. Si no aparece warning, hay que explicar por qué la construcción sería incorrecta igualmente.

PENDIENTE DE EVIDENCIA REAL: Adjuntar extractos reales de logs de LibreLane (archivo, línea y explicación), diferenciando entre error detectado y adaptación preventiva.

3. MEMORIA Y PROGRAMA DEL JUEGO

3.1. Uso real del programa

El assembler genera 153 palabras de instrucciones (612 bytes). El análisis estático de las instrucciones lw/sw del juego indica que los accesos de datos corresponden a periféricos mapeados en memoria, por lo que se detectaron cero palabras de RAM de datos utilizadas en este juego. Para permitir pruebas con otros programas reservamos ocho palabras de RAM. Esa reserva no debe confundirse con datos efectivamente utilizados.

La cifra de 153 proviene del programa ensamblado. El conteo dinámico de accesos a datos debe confirmarse con el log de simulación si se quiere presentarlo como medición, no solo como análisis estático.

3.2. Dos alternativas que se comparan

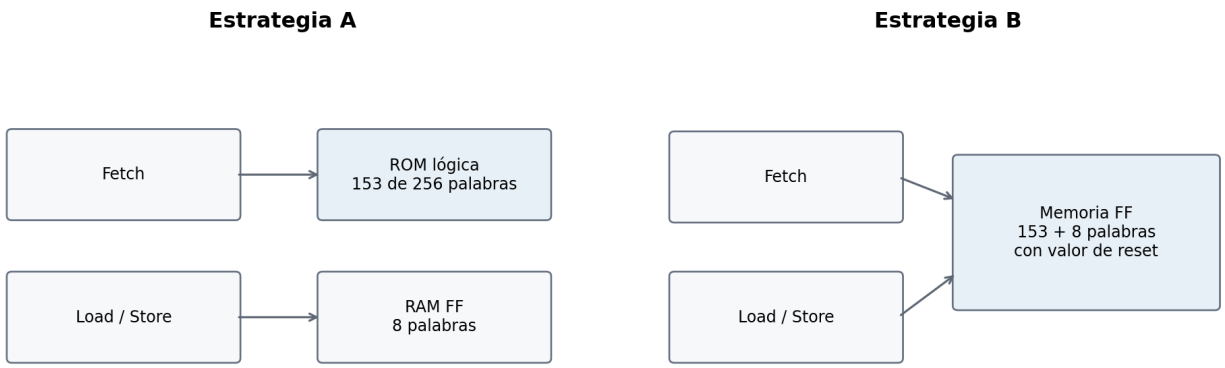


Figura 2. Memoria A y B: la diferencia principal es almacenar instrucciones como función lógica o como bits de flip-flops.

Característica	A: ROM + RAM FF	B: memoria unificada FF
Programa	ROM combinacional generada por assembler	FF con patrón inicial por reset
Instrucciones	153 palabras; capacidad ROM hasta 256	153 palabras dentro de memoria unificada
Datos reservados	8 palabras FF desde 0x00000400	8 palabras FF junto al programa
Lecturas	Salida de ROM registrada	Dos puertos de lectura registrados
Software game.s	Mismo programa	Mismo programa
Limitación	Puerto de datos no lee la ROM	Cada bit de programa requiere FF físico

3.3. Hipótesis y métricas de las dos memorias

Antes de las corridas, esperamos que la estrategia B tenga más área y flip-flops porque almacena explícitamente el programa. Las notas del repo incluyen estimaciones locales de síntesis, útiles para anticipar la diferencia, pero insuficientes para afirmar un resultado físico final.

Métrica solicitada	A	B
Área de celdas (µm²)	PENDIENTE	PENDIENTE
Flip-flops	PENDIENTE	PENDIENTE
Utilización (%)	PENDIENTE	PENDIENTE
Menor período cerrado (ns)	PENDIENTE	PENDIENTE
Frecuencia máxima (MHz)	PENDIENTE	PENDIENTE

Referencia de estimación pre-layout (NO resultado final): A ≈ 162.146 µm² y 1.070 FF; B ≈ 881.309 µm² y 5.967 FF, según docs/notas_informe.md. Esos valores deben sustituirse por métricas reales de LibreLane y acompañarse de su ruta de reporte.

4. SÍNTESIS Y TRAZABILIDAD JERÁRQUICA

La configuración busca conservar las instancias espino_register_file, espino_alu y pochoco_cycle_counter para poder explicar de dónde viene el área y ubicarlas después en el layout. Cuando se optimiza o aplana una jerarquía, el análisis se reconstruye con estadísticas por módulo o agrupando las celdas de cada instancia; el procedimiento está apoyado por asic/scripts/area_by_module.py.

4.1. Cinco celdas más usadas

Celda estándar real	Cantidad	Función que explica su uso
PENDIENTE	—	Obtener de reporte final de síntesis
PENDIENTE	—	Obtener de reporte final de síntesis
PENDIENTE	—	Obtener de reporte final de síntesis
PENDIENTE	—	Obtener de reporte final de síntesis
PENDIENTE	—	Obtener de reporte final de síntesis

4.2. Área por bloque

Bloque	Área (µm²)	% del total
Register file	PENDIENTE	PENDIENTE
ALU	PENDIENTE	PENDIENTE
Contador de ciclos	PENDIENTE	PENDIENTE
Total	PENDIENTE	100 %

La hipótesis del equipo es que el banco de registros será uno de los mayores consumidores de celdas, ya que contiene registros y multiplexores para las dos lecturas. El reporte final debe señalar cuál módulo ocupa más área y comparar el resultado observado con esta hipótesis.

PENDIENTE DE EVIDENCIA REAL: Guardar stat.rpt o métricas jerárquicas y documentar método de atribución de área.

5. TEMPORIZACIÓN, RUTA CRÍTICA Y CÁLCULOS

5.1. Configuración de timing y criterio de cierre

El período objetivo es de 100 ns (10 MHz). Se debe probar al menos tres períodos, incluyendo el de referencia. Una corrida se considera válida cuando WNS ≥ 0 y TNS = 0 en setup y hold para el corner típico y el más lento, además de los demás corners exigidos. Es importante que un CLOCK_PERIOD diferente puede cambiar la síntesis, los buffers, el placement y por tanto el área.

Período ns	Setup TT WNS/TNS	Hold TT WNS/TNS	Setup lento WNS/TNS	Hold lento WNS/TNS	Cierra
100	PENDIENTE	PENDIENTE	PENDIENTE	PENDIENTE	—
80 (por medir)	PENDIENTE	PENDIENTE	PENDIENTE	PENDIENTE	—
60 (por medir)	PENDIENTE	PENDIENTE	PENDIENTE	PENDIENTE	—

Se sugiere localizar el menor período aprobado usando el barrido automático; los valores 80 y 60 ns son puntos de exploración, no resultados ya medidos.

5.2. Ruta crítica

Se debe informar celda de inicio y fin, módulo y etapa microarquitectónica de Espino asociada: fetch, decode, ejecución, memoria o write-back. El script asic/scripts/crit_path.py apoya la extracción, pero la ruta debe corresponder a un reporte concreto de la corrida.

Inicio	Fin	Etapa de Espino	Slack de la ruta
PENDIENTE	PENDIENTE	PENDIENTE	PENDIENTE

5.3. Optimización evaluada

Se implementó la variante ESPINO_REG_DECODE que registra la decodificación en el ciclo de espera de lectura del register file. La hipótesis es sacar lógica combinacional del camino de ejecución sin cambiar la duración funcional del juego. Para demostrar la mejora hay que comparar strategy_A y strategy_A_opt con el mismo contexto de constraints.

Comparación	Base A	A optimizada
Área de celdas	PENDIENTE	PENDIENTE
Setup WNS más lento	PENDIENTE	PENDIENTE
Hold WNS más lento	PENDIENTE	PENDIENTE
Período mínimo cerrado	PENDIENTE	PENDIENTE

5.4. Constantes del juego al pasar de 25 a 10 MHz

El juego usa un contador de ciclos para medir tiempos. En la Go Board cada ciclo dura 40 ns (25 MHz). Con T = 100 ns, la frecuencia pasa a 10 MHz. Para mantener la fase de 3 s y el display en décimas, cambiamos CLK_HZ y dejamos que el assembler recalcule el resto.

Constante	FPGA 25 MHz	ASIC 10 MHz	Uso
CLK_HZ	25.000.000	10.000.000	Ciclos por segundo
TENTH	2.500.000	1.000.000	0,1 s

HALF_TENTH	1.250.000	500.000	Redondeo
ALL_ON_TIME	75.000.000	30.000.000	Fase de 3 segundos
DEBOUNCE	500.000	200.000	20 ms
MAX_CYCLES	247.500.000	99.000.000	Límite de 9,9 s
Divisor promedio	25.000.000	10.000.000	10 rondas en décimas

Comprobación: 3 s × 10.000.000 ciclos/s = 30.000.000 ciclos. Una décima equivale a 1.000.000 ciclos. El cambio se realiza con make asic-gen, que vuelve a generar el programa desde game.s.

6. FLOORPLAN, PLACEMENT Y COSTO

La configuración base usa FP_CORE_UTIL = 40 %. Se explorarán al menos tres objetivos de utilización o tres tamaños de die y se elegirá la menor área con flujo cerrado. No basta con escoger el die geométricamente más pequeño: también debe cumplirse timing, signoff y condiciones de congestión.

Utilización objetivo	Área die mm²	Congestión / violaciones	Cumple
30 %	PENDIENTE	PENDIENTE	—
40 % (base)	PENDIENTE	PENDIENTE	—
50 %	PENDIENTE	PENDIENTE	—

PENDIENTE DE EVIDENCIA REAL: Insertar gráfico generado desde corridas del sweep: utilización frente a congestión o violaciones reportadas.

6.1. Terminales del macro

El archivo asic/pin_order.cfg agrupa terminales según función. Reloj y reset se separan de los buses de LEDs, botones, segmentos y SPI para facilitar la conexión con otros bloques. Estos terminales no reemplazan los pads externos que necesitaría un chip encapsulado.

6.2. Costo equivalente didáctico

La pauta propone US$23.000 por 1 mm² suponiendo relación lineal. Por tanto: costo equivalente = área del diseño en mm² × 23.000 US$/mm². Si el die midiera 0,42 mm², por ejemplo, el valor sería aproximadamente US$9.660. Esto es solo un ejemplo matemático; falta usar el área real del layout y no incluye yields, pads, empaquetado ni costos fijos.

7. SIGNOFF Y LAYOUT

Para cerrar la entrega se exige DRC y LVS limpios, violaciones de antena resueltas o justificadas y timing cerrado en corners. Los resultados finales tienen que venir de logs reales. La carpeta docs/evidencia se dejó destinada a estos archivos.

Comprobación	Resultado final	Evidencia
DRC	PENDIENTE	Ruta/captura del reporte
LVS	PENDIENTE	Ruta/captura del reporte
Antena	PENDIENTE	Conteo y justificación, si aplica
GDSII generado	PENDIENTE	Nombre del archivo / run tag

7.1. Identificación visual de bloques

En KLayout u OpenROAD GUI se debe marcar el register file, la ALU y el contador de ciclos. El informe debe indicar cómo se ubicaron las instancias, usando el nombre jerárquico preservado o la trazabilidad documentada en la sección 4.

PENDIENTE DE EVIDENCIA REAL: Insertar captura anotada del layout real; no utilizar un diagrama arquitectónico en lugar del GDSII.

8. SIMULACIÓN FUNCIONAL POST-SÍNTESIS

El testbench sim/asic/tb_asic.v usa el mismo conjunto de pulsaciones para RTL y netlist sintetizado, con instantes dados en ciclos. Para acelerar la simulación se genera una versión del juego con CLK_HZ = 20.000, manteniendo idénticas las constantes y entradas en ambas configuraciones.

La prueba debe evidenciar dos comportamientos: una respuesta correcta produce el mismo conteo de ciclos y una pulsación incorrecta provoca EE y el reinicio de la ronda actual, sin perder las correctas. Las notas de trabajo reportan coincidencia en corridas locales anteriores, pero este paquete no incluye los logs finales correspondientes, así que se dejan como pendientes de respaldo.

Caso	RTL	Netlist post-síntesis	Archivo de prueba
Respuesta correcta	PENDIENTE	PENDIENTE	PENDIENTE
Pulsación incorrecta	PENDIENTE	PENDIENTE	PENDIENTE
Ronda reiniciada	PENDIENTE	PENDIENTE	PENDIENTE

Como referencia para revisar el comportamiento, las notas anteriores mencionan 14.349 ciclos en una respuesta correcta y 8.778 en otra, ambos coincidentes entre RTL y netlist. No deben presentarse como evidencia de la corrida final sin adjuntar las trazas correspondientes.

9. REPRODUCIBILIDAD Y ENTORNO

9.1. Configuración obligatoria

Parámetro	Valor
PDK configurado	gf180mcuD
Biblioteca estándar configurada	gf180mcu_fd_sc_mcu7t5v0
CLOCK_PERIOD	100 ns
FP_CORE_UTIL	40 %
Top-level	asic_top

El README deja preparado un flujo con LibreLane, generación de memorias, síntesis, barridos y scripts de métricas. La versión real de LibreLane y la revisión exacta del PDK tienen que recogerse desde el entorno usado en la corrida final; no basta con declarar la versión para la que se preparó la configuración.

```
make check-local
make asic-gen
make asic-A
make asic-B
make asic-A-opt
bash asic/scripts/versions.sh | tee docs/versiones_capturadas.txt
```

PENDIENTE DE EVIDENCIA REAL: Completar versiones exactas de LibreLane, PDK, Yosys, OpenROAD, Magic, Netgen y KLayout y anexar la salida real del script.

10. CONCLUSIONES PROVISIONALES

Separar el código común del específico de FPGA facilitó adaptar Pochoco SoC sin rehacer el procesador. Las diferencias relevantes fueron la memoria física, el reset y los terminales del diseño; en el ASIC ya no se puede depender de la inicialización implícita de las BRAM al programar la placa.

La comparación entre ROM combinacional y memoria de FF es la decisión de arquitectura que esperamos que impacte más el área. El diseño deja preparada una medición justa con el mismo juego y las mismas restricciones. Sin embargo, la conclusión cuantitativa debe escribirse después de obtener las corridas finales.

También se implementó una variante de decodificación registrada para explorar la ruta crítica y su costo en celdas. El proyecto quedará realmente terminado cuando podamos mostrar con números el mínimo período cerrado, el área mínima válida, los resultados DRC/LVS/antena y la equivalencia RTL/post-síntesis.

ANEXO A. ARCHIVOS QUE RESPALDAN EL INFORME

Ruta	Uso
docs/notas_informe.md	Estudio previo y estimaciones, no signoff
asic/strategy_A/config.yaml	ROM + RAM, 100 ns y 40 %
asic/strategy_B/config.yaml	Memoria unificada FF
asic/strategy_A_opt/config.yaml	Variante de decodificación registrada
asic/scripts/sweep.py	Barridos de timing y utilización
asic/scripts/metrics.py	Resumen de métricas y criterio de cierre
asic/scripts/crit_path.py	Extracción de ruta crítica
asic/scripts/area_by_module.py	Área con trazabilidad jerárquica
sim/asic/tb_asic.v	Pruebas del juego sobre RTL / netlist

ANEXO B. EVIDENCIA NECESARIA ANTES DE ENTREGAR

☐ Log y línea de cada incompatibilidad del RTL FPGA o explicación si no hubo warning.
☐ Reporte de síntesis A y B con área, FF, utilización y frecuencia máxima.
☐ Cinco celdas más frecuentes; áreas del register file, ALU y contador.
☐ Barrido de por lo menos tres períodos con setup y hold por corner.
☐ Ruta crítica real y medición antes/después de optimización.
☐ Tres alternativas de utilización/die y gráfico de congestión o violaciones.
☐ DRC, LVS, antena y captura de layout con instancias anotadas.
☐ Logs RTL y post-síntesis con los mismos estímulos en ciclos.
☐ Versiones exactas del entorno y revisión del PDK en el README.